HDOC Day-63 Activity

Finding the Missing Number Using Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-63

Aim

To write a C program that finds the one missing integer from the range **0 to n** when the array contains n distinct values from that range. The array is passed to **missing_number()** using its address and accessed through pointers.

Problem Requirements

- Read the size and elements of a variable-length integer array **nums[]**.
- The array contains distinct integers from 0 to size with exactly one number missing.
- Pass the array to **missing_number()** using its address.
- Find and display the missing integer.
- Use the fact that n numbers are present from a range containing n+1 possible values.

Method Used

The complete range is 0, 1, 2, ..., n. Its expected sum is **n(n+1)/2**. If we subtract the sum of all elements actually present in nums[], the remaining value is exactly the missing number.

Algorithm

1. Read the array size n.
2. Read n distinct integers into nums[].
3. Pass nums and n to **missing_number(nums, n)**.
4. Calculate **expected_sum = n * (n + 1) / 2**.
5. Initialize actual_sum = 0 and traverse the array using pointer notation.
6. Add each value ***(nums + i)** to actual_sum.
7. Calculate **missing = expected_sum - actual_sum**.
8. Display the missing number.

C Program

```c
#include <stdio.h>

void missing_number(int *nums, int n)
{
    int expected_sum = n * (n + 1) / 2;
    int actual_sum = 0;

    for (int i = 0; i < n; i++)
    {
        actual_sum = actual_sum + *(nums + i);
    }

    int missing = expected_sum - actual_sum;

    printf("Missing number: %d", missing);
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    missing_number(nums, n);

    return 0;
}
```

Pointer Usage

missing_number(nums, n) passes the address of the first array element. The parameter **int *nums** receives that address. The expression ***(nums + i)** accesses nums[i] through pointer arithmetic.

Example 1 Dry Run

Item	Calculation	Value
Size n	n = 3	3
Expected sum	3 x 4 / 2	6
Actual sum	3 + 0 + 1	4
Missing number	6 - 4	2

The complete range is 0, 1, 2, 3. The array contains 3, 0, 1, so the missing value is **2**.

Test Cases / Expected Results

Case	Size	Input Array	Expected Output

Example 1	3	3 0 1	Missing number: 2
Example 2	2	0 1	Missing number: 2
Example 3	9	9 6 4 2 3 5 7 0 1	Missing number: 8
Example 4	1	0	Missing number: 1

Example 2 Explanation

For size 2, the complete range is 0, 1, 2. The array contains 0 and 1, so **2** is missing.

Example 3 Explanation

For size 9, the expected sum of 0 through 9 is 45. The given array sums to 37. Therefore, **45 - 37 = 8**, so the missing number is 8.

Example 4 Explanation

For size 1, the possible values are 0 and 1. Since the array contains only 0, the missing number is **1**.

Complexity

Time Complexity: O(n), because the array is traversed once.

Space Complexity: O(1) extra space because only a few integer variables are required.

Result

The program successfully finds the missing integer from the range 0 to n using pointer-based array access. It computes the expected range sum and subtracts the sum of the given distinct elements to obtain the missing number.

Execution Example

```
Enter the size of array: 3
Enter the elements: 3 0 1
Missing number: 2
```